

UNIT-IV: DATA WRANGLING

PYTHON PACKAGES FOR DATA WRANGLING

Array in Data Science:

Array is a row of data.

For ^{large} dataset the calculation of variance, standard deviation for large dataset is a datawrangling, hassle. For finding the data so we go for wrangling.

Numpy:

For performing statistical operation for large dataset we use numpy.

Eg:- Variance for n-value more than 100

Using numpy to find the hours, minutes & seconds.

Pandas:

For data visualization purpose, presentation of the table.

Array:

For all the row we consider as an array in numpy. Each numpy array consist of

A[0]	A[1]	A[2]	A[3]	A[4]
0	0.5	1	3	5

1) python ~~point~~ object Head.

2) Data

3) Dimensions.

4) Strides.

python object Head	Data
Dimension	Strides

import numpy as np.

arr = np.array([2, 4, 6, 8, 10])

Print("No. of dimensions")

Print("Shape of arr" arr.shape)

Print("Size of array" arr.size)

Print("Array is a type of")

arr.dtype)

Numpy Arrays:-

Step 1: Import the numpy package.

syn:- import numpy as np.

Step 2: To create an array.

syn:- array name = np.array [1, 2, 3, 4]. ^{Size}

Attributes of Numpy array:-

size, dimensions, datatype, shape

Array Declaration:-

import numpy as np.

x1 = np.array([45, 89, 64, 33]) # 1-D array

x2 = np.array([[1, 2, 3], [4, 5, 6]]) # 2-D array

print(x1)

print(x2)

1-D array:-

Syntax:- Array name = np.array([el1, el2])

2-D array:-

Syntax: Array Name = np.array([[el1, el2], [el1, el2]])

O/P:- [45, 89, 64, 33].

[[1, 2, 3]

[4, 5, 6]].

✓

Numpy has many Built-in function or pre-defined codes for performing a certain operation.

Attributes of array:-
ndim, size, shape.

shape is used to find out the shape of the array.

Eg:- 1D, 2D.

n-dim: To find out dimensionality.

Syntax:-

arrayname.methodname.

Program:

```
import numpy as np
x2 = np.array([1, 2, 3], [4, 5, 6], [7, 8, 9])
print("x2 ndim:", x2.ndim)
```

```
print("x2 shape:", x2.shape)
```

```
print("x2 size:", x2.size)
```

o/p:-

```
[[1, 2, 3]
```

```
[4, 5, 6]
```

```
[7, 8, 9]]
```

```
x2.ndim: 2
```

```
x2.shape: (3, 3)
```

```
x2.size: 9
```



```
x3 = np.array([5.5, 6.9])
```

```
print(x3)
```

```
print(x3.dtype)
```

O/p:- float64.

Itemsize:

Memory occupies in the system.

```
print(x2, itemsize)
```

```
print(x2, nbytes)
```

O/p: 8

72

Array Indexing:

In a one-dimensional array, the i^{th} value can be accessed by specifying the desired:

Index in square brackets, just as with python lists:

1-Dimensional array

```
[ ] x1:
```

```
array([45, 89, 64, 33])
```

// Array name[size] -- syntax for indexing

```
x1[-2]
```

O/p:- 64.

2-Dimensional array:

```
[ ] array([[1, 2, 3],
```

```
[4, 5, 6]
```

```
[7, 8, 9]])
```

array(row, column) -- syntax indexing

```
x2[1, 2] = 60.
```

$[] \times 2$
o/p: array([[1, 2, 3],
[4, 5, 6],
[7, 8, 9]])

$X[0, 0] = 12$

$X \times 2$
o/p: array([[12, 5, 2, 4],
[7, 6, 8, 4],
[1, 6, 7, 7]])

Array Slicing:-

We can use square brackets to access individual array elements, we can also use them to access subarrays with the slice notation, marked by the (:) character.

$X[\text{start}:\text{stop}:\text{step}]$

Eg:- # array name = np.arange(stop)
 $X = \text{np.arange}(10)$

o/p: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
 $X[5:9]$

o/p: array([5, 6, 7, 8])

$X[:5]$ #. first 5 elements.

o/p: array([0, 1, 2, 3, 4])

X[4:7]
o/p: array([4, 5, 6])

Eg:-

[]x

o/p: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

arrayname [start::stop]

X[1::3] # every other element, starting at index 1

o/p: array([1, 4, 7])

Multi-dimensional subarrays

[] x 2

array([[1, 2, 3],
[4, 5, 6],
[7, 8, 9]])

arrayname [start row: stop col, row: col]

x2[0:2, 0:2] # two rows, three columns

o/p: array([[1, 2],
[4, 5]])

Accessing array rows & columns:-

[] x 2

array([[1, 2, 3],
[4, 5, 6],
[7, 8, 9]])

print(x2[:, 1]) # first column of x2

o/p: [2, 5, 8]

print(x2[2, :]) # first row of x2

o/p: [7 8 9]

Creating copies of arrays:-

subcopy array[row, col] = element to be changed

X2-sub-copy[0,0]=8.

print(X2-sub-copy) # PURE COPY.

O/p:- [[8 2 3]

[4 5 6]

[7 8 9]]

Reshaping arrays:-

X = np.array([1, 2, 3, 4])

row vector via reshape

array.name.reshape(row, col)

X.reshape((2, 2))

O/p: array([[1, 2],

[3, 4]])

Concatenation of arrays:-

concatenation, or joining of two arrays in Numpy, is primarily accomplished

using the routines np.concatenate, np.vstack,

& np.hstack. np.concatenate takes a tuple or list in the first argument.

Eg:- import numpy as np:

X = np.array([1, 2, 3])

Y = np.array([8, 12, 14])

print(np.concatenate([X, Y]))

O/p:- array([1, 2, 3, 8, 12, 14])

```
[ ] z = [99, 99, 99]
```

```
print(np.concatenate([x, y, z]))
```

O/p: [1 2 3 8 12 14 99 99 99]

✓ Splitting of arrays:-

The opposite of concatenation is splitting, which is implemented by the functions `np.split`, `np.hsplit`, `np.vsplit`.

Eg: Import numpy as np.

```
x = [1, 2, 3, 99, 99, 3, 2, 1]
```

```
array name - np.split(array name, index)
```

```
x1, x2 = np.split(x, [2])
```

```
print(x1, x2)
```

O/p:

```
[1 2] [3 99 99 3 2 1]
```

COMPUTATIONAL AND AGGREGATION:-

computation on array and aggregation:-

Computation on Numpy arrays can be very fast, or it can be very slow. The key to making it fast is to use vectorized operations, generally implemented through Numpy's universal functions (Ufuncs).

slowness of loops:-

```
[ ] a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
def add_constant(a):
```

```
    for i in range(len(a)):
```

```
        a[i] = a[i] + 5
```

```
    return a
```

Timeit add_constant(a)

o/p: $1.22 \mu s \pm 55.6$ ns per loop (mean $\pm$ std.
of 7 runs, 100000 loops each).

Introducing UFuncs:-

```
import numpy as np
```

```
a = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
```

```
%timeit a+5
```

o/p: 788 ns ± 13.3 ns per loop (mean $\pm$ std.
of 7 runs, 1000000 loops each).

Exploring Numpy's UFuncs.

Ufuncs exist in two flavors: unary
ufuncs, which operate on a single input,
and binary ufuncs, which operate on two
inputs.

Array arithmetic

The standard add, sub, mul, div
can all be used:

```
[ ] x = np.arange(4)
```

```
print("x = ", x)
```

```
print("x + 5 = ", x + 5)
```

```
print("x - 5 = ", x - 5)
```

```
print("x * 2 = ", x * 2)
```

```
print("x / 2 = ", x / 2)
```

```
print("x // 2 = ", x // 2)
```

O/p:-

$$X = [0 \ 1 \ 2 \ 3]$$

$$X+5 = [25 \ 26 \ 27 \ 28]$$

$$X-5 = [-5 \ -4 \ -3 \ -2]$$

$$X * 2 = [0 \ 2 \ 4 \ 6]$$

$$X / 2 = [0. \ 0.5 \ 1. \ 1.5]$$

$$X // 2 = [0 \ 0 \ 1 \ 1]$$

There is also a unary ufunc for negation, and a ** operator for exponentiation, and a % operator for modulus.

[] print(x)

print("-x = ", -X)

print("X ** 2", X ** 2)

print("X % 3", X % 3)

O/p: - [0 1 2 3]

$$-X = [-0 \ -1 \ -2 \ -3]$$

$$X ** 2 = [0 \ 1 \ 4 \ 9]$$

$$X \% 3 = [0 \ 1 \ 2 \ 0]$$

The following table list the arithmetic operators:-

Operator	Equivalent ufunc	Description
+	np.add	Addition (eg: , 1 + 1 = 2)
-	np.sub	subtraction (eg: , 3 - 2 = 1)
-	np.negative	Unary negation (eg: , -2)
*	np.multiply	Multiplication (eg: , 2 * 3 = 6)
/	np.divide	Division (eg: , 3 / 2 = 1.5)
//	np.floor-divide	Floor Division (eg: , 3 // 2 = 1)
**	np.power	Exponentiation (eg: , 2 ** 3 = 8)
%	np.mod	Modulus / remainder (eg: , 9 % 4 = 1).

2

Aaange (1):

arange is used to return an array of element within the size passed.

 $x + 2$

```
np.add(x, 2)
```

o/p: array([2, 3, 4, 5]).

Absolute value:

```
x = np.array([-2, -1, 0, 1, 2])
```

```
np.absolute(x)
```

o/p: array([2, 1, 0, 1, 2]).

Trigonometric functions:

```
theta = np.array([0.157, 3.14])
```

```
print("theta =", theta)
```

```
print("sin(theta) =", np.sin(theta))
```

```
print("cos(theta) =", np.cos(theta))
```

```
print("tan(theta) =", np.tan(theta))
```

o/p:

```
theta = [0.157 3.14]
```

```
sin(theta) = [0.699999 0.00159625]
```

```
cos(theta) = [1.000000e+00 -7.96326711e-04]
```

```
tan(theta) = [0.000000e+00 -1.5926549e-03]
```


Exponentials & logarithms.

$x = [1, 2, 3]$

`print("x = ", x)`

`print("e^x = ", np.exp(x))`

`print("3^x = ", np.power(3, x))`

O/p:

$x = [1, 2, 3]$

$e^x = [2.71 \quad 7.38 \quad 20.0855]$

$3^x = [3 \quad 9 \quad 27]$

logarithms

$x = [1, 2, 4, 10]$

`print("x = ", x)`

`print("log(x) = ", np.log(x))`

`print("log2(x) = ", np.log2(x))`

O/p:

$x = [1, 2, 4, 10]$

$\ln(x) = [0. \quad 0.69 \quad 1.38 \quad 2.30]$

$\log_2(x) = [0. \quad 0.30 \quad 0.60 \quad 1.]$

Aggregation

`import numpy as np`

`L = np.array([1, 2, 3])`

`np.sum(L), np.min(L), np.max(L)`

O/p: (6, 1, 3).

2.

One dimensional Aggregation:

```
L = np.array([[1, 2, 3],
              [4, 5, 6],
              [8, 9, 10]])
```

axis = 0 → row
axis = 1 → column

```
np.sum(L), np.min(L), np.max(L)
```

O/p: (48, 1, 10)

np.sum	np.nansum	compute sum of elements
np.prod	np.nanprod	compute product of elements
np.mean	np.nanmean	compute mean of elements
np.std	np.nanstd	compute standard deviation
np.var	np.nanvar	compute variance
np.min	np.nanmin	Find minimum value
np.max	np.nanmax	Find maximum value.

```
heights = np.array([160, 170, 158, 190, 168])
print("Mean Height:", heights.mean())
print("Standard deviation:", heights.std())
print("Minimum height:", heights.min())
print("Maximum height:", heights.max())
```


comparisons, Masks, and Boolean Logic.

```
import numpy as np
```

```
x = np.array([1, 2, 3, 4, 5])
```

```
x < 4 # less than
```

```
o/p: array([ True,  True,  True, False, False])
```

```
x > 3 # Greater than
```

```
o/p: array([ False, False, False,  True,  True])
```

```
x == 3 # equal
```

```
o/p: array([ False, False,  True, False, False])
```

22/10/22 -
CWIIT-5
Completed

pandas - part - II, (Join).

concat and append:

combining two tables of data
to gather useful information from the resultant
table.

Eg: i) two tables of student details are
present. ii) Based on one parameter we can
join two tables together.
iii) For large dataset combining tables

to play a major role. for easier analysis

Step 1: create arrays.

For combining two array using
np.concatenate

Syntax: np.concatenate([array name])

Simple concatenation with pd.concat

pandas has a function, pd.concat() which has a similar syntax np.concatenate

```
ser1 = pd.Series(['A', 'B', 'C'], index=[1, 2, 3])
```

```
ser2 = pd.Series(['D', 'E', 'F'], index=[4, 5, 6])
```

```
pd.concat([ser1, ser2])
```

```
1    A
```

```
2    B
```

```
3    C
```

```
4    D
```

```
5    E
```

```
6    F
```

Creating the DataFrames.

```
df1 = pd.DataFrame({'A': ['A0', 'A1', 'A2', 'A3'],
```

```
                      'B': ['B0', 'B1', 'B2', 'B3']})
```

```
df2 = pd.DataFrame({'A': ['A4', 'A5', 'A6', 'A7'],
```

```
                      'B': ['B4', 'B5', 'B6', 'B7']})
```


df1 (columns) arrays : example

	A	B
0	A0	B0
1	A1	B1
2	A2	B2
3	A3	B3
df2	A	B
4	A4	B4
5	A5	B5
6	A6	B6
7	A7	B7

(rows) arrays : example

	A	B
0	A0	B0
1	A1	B1
2	A2	B2
3	A3	B3
4	A4	B4
5	A5	B5
6	A6	B6
7	A7	B7

pd.concat([df1, df2], axis=1) $\Rightarrow$ column wise concatenation

step 1: concatenation of two arrays.

step 2: pd.concat, pd-concate

step 3: 0 means row wise

1 means column wise:

O/p:

	A	B
0	A0	B0
1	A1	B1
2	A2	B2
3	A3	B3

	A	B
4	A4	B4
5	A5	B5
6	A6	B6
7	A7	B7

pd.concat([df1, df2], axis=1)

	A	B
0	A0	B0
1	A1	B1
2	A2	B2
3	A3	B3
4	A4	B4
5	A5	B5
6	A6	B6
7	A7	B7

Syntax: dataframe1.append(dataframe2)

df1.append(df2)

	A	B
0	A0	B0
1	A1	B1
2	A2	B2
3	A3	B3
4	A4	B4

Ignore Indexing:-

When appending two tables duplicate indices appear in the resultant table:-

To avoid such index we use to boolean Index

	A	B
0	A0	B0
1	A1	B1
2	A2	B2
3	A3	B3

	A	B
0	A4	B4
1	A5	B5
2	A6	B6
3	A7	B7

	A	B
0	A0	B0
1	A1	B1
2	A2	B2
3	A3	B3
0	A4	B4
1	A5	B5

`pd.concat([df1, df2], ignore_index=True)`

o/p:

	A	B
0	A ₀	B ₀
1	A ₁	B ₁
2	A ₂	B ₂
3	A ₃	B ₃
4	A ₄	B ₄
5	A ₅	B ₅
6	A ₆	B ₆
7	A ₇	B ₇

concatenation with Joins:-

It is used for appending or combining tables based on a particular column name.

Types:

1. Inner Join $\Rightarrow$ The additional name will not appear in the resultant table.
2. Left Join
3. Right Join
4. Full Outer.

Inner join - only common for both tables.

`pd.merge(df1, df2, on='Name', how='inner')`

The additional names which are not common will not appear in the resultant table.

df1

	Name	RollNo	Marks
0	Arun	1	80
1	Vijay	2	75
2	Gopal	3	25
3	Gowri	4	35
4	Govindh	5	79

df2

	Name	Dept.
0	Arun	CSE
1	Vijay	IT
2	Gopal	CSE
3	Gowri	MECH
4	Govindh Krishna	AUTO

O/P:

	Name	RollNo	Marks	Dept.
0	Arun	1	80	CSE
1	Vijay	2	75	IT
2	Gopal	3	25	CSE
3	Gowri	4	35	MECH
4	Govindh	5	79	AUTO
5	Krishna	6		

Left Join:

Left Join - return all rows of first/left table
pd.merge(df1, df2, on='Name', how='left')

df1

	Name	RollNo	Marks	Dept
0	Arun	1	80	CSE
1	Vijay	2	75	IT
2	Gopal	3	25	CSE
3	Gouri	4	35	MECH.
A.	Govindh Krishna	5 NAN	79 NAN	NAN AUTO.

Right Join:

Right Join - return all rows of second/right table
pd.merge(df1, df2, on='Name', how='right')

	Name	RollNo	Marks	Dept.
0	Arun	1	80	CSE
1	Vijay	2	75	IT
2	Gopal	3	25	CSE
3	Gouri	4	25	MECH
4	Krishna	NAN	NAN	AUTO.

OUTER

JOIN

pd.merge(df1, df2, on='Name', how='outer')

	Name	RollNo	Marks	Dept
0	Arun	1	80	CSE
1	Vijay	2	75	IT
2	Gopal	3	25	CSE
3	Gouri	4	35	MECH
4	Govindh	5	79	NAN
5	Krishna	NAN	NAN	AUTO